Tài liệu mã nguồn và cơ sở lý thuyết

Dự án GARCH-VN30F1M-MLE-VB-MCMC: so sánh họ mô hình GARCH trên hợp đồng tương lai VN30F1M
bằng ước lượng hợp lý cực đại, Bayes biến phân và MCMC

Nguyễn Hoài Nam và cộng sự

Tháng 9 năm 2026

Tóm tắt

Tài liệu mô tả mã nguồn của đề tài: cách cài đặt và chạy, các thư viện được dùng, cấu trúc thư mục, vai trò của từng tệp và từng hàm, cùng cơ sở toán học của mọi kỹ thuật được cài đặt. Mã nguồn gồm thư viện Python `garch_vn30` và các script điều phối ba bước nghiên cứu: (1) so sánh sáu mô hình GARCH/EGARCH một trạng thái bằng ba phương pháp suy diễn trên dữ liệu VN30F1M; (2) nghiên cứu mô phỏng khôi phục tham số cho EGARCH-X Student-t; (3) lấy mẫu MCMC cho mô hình thắng cuộc. Ba phương pháp suy diễn dùng chung một hàm log-hợp lý, nên khác biệt giữa kết quả chỉ đến từ phương pháp và tiên nghiệm, không đến từ cách viết mô hình.

Mục lục

1 Tổng quan **2**
1.1 Mục tiêu và thiết kế ba bước . . . 2
1.2 Nguyên tắc cài đặt . . . 2

2 Cài đặt và cách chạy **2**
2.1 Yêu cầu phần mềm và thư viện . . . 2
2.2 Cài đặt . . . 3
2.3 Chạy từng bước . . . 3

3 Cấu trúc thư mục và luồng dữ liệu **4**

4 Cơ sở lý thuyết **4**
4.1 Dữ liệu và biến số . . . 5
4.2 Phân phối sai số . . . 5
4.3 Mô hình GARCH(1,1) và GARCH-X . . . 5
4.4 Mô hình EGARCH(1,1) và EGARCH-X . . . 6
4.5 Hàm log-hợp lý có điều kiện . . . 6
4.6 Danh mục mô hình . . . 6
4.7 Ước lượng hợp lý cực đại . . . 6
4.8 Mô hình Bayes và tiên nghiệm . . . 7
4.9 Bayes biến phân . . . 8
4.10 MCMC bằng NUTS . . . 8

4.11 Đánh giá ngoài mẫu . . . 9
4.12 Kiểm định thống kê ban đầu . . . 10
4.13 Nghiên cứu mô phỏng khôi phục tham số . . . 10

5 Mô tả chi tiết từng tệp và hàm **11**
5.1 Gói `garch_vn30` (`__init__.py`) . . . 11
5.2 `du_lieu/doc_du_lieu.py` — đọc dữ liệu phút . . . 11
5.3 `du_lieu/chuan_bi.py` — chuẩn bị dữ liệu mô hình . . . 12
5.4 `kiem_dinh/kiem_dinh_so_bo.py` — kiểm định ban đầu . . . 12
5.5 `mo_hinh/phan_phoi.py` — phân phối sai số . . . 12
5.6 `mo_hinh/garch.py` và `mo_hinh/egarch.py` — phương trình mô hình . . . 12
5.7 `mo_hinh/danh_muc.py` — danh mục mô hình . . . 13
5.8 `suy_dien/mle.py` — ước lượng hợp lý cực đại . . . 13
5.9 `suy_dien/bayes.py` — mô hình Bayes . . . 13
5.10 `suy_dien/vb.py` và `suy_dien/mcmc.py` . . . 13
5.11 `danh_gia/` — đánh giá ngoài mẫu . . . 14
5.12 `mo_phong/` — nghiên cứu mô phỏng . . . 14
5.13 Các script trong `scripts/` . . . 14

6 Hướng dẫn đọc mã nguồn **15**
6.1 Quy ước chú thích trong mã . . . 15
6.2 Thứ tự đọc đề nghị . . . 15

7 Các hàm thư viện bên ngoài được sử dụng **15**

8 Ví dụ: một lần chạy từ dữ liệu đến kết quả **17**

9 Định dạng tệp kết quả **18**

10 Tính tái lập và kiểm chứng **18**

11 Tóm tắt kết quả hiện có **19**

1 Tổng quan

1.1 Mục tiêu và thiết kế ba bước

Đề tài mô hình hóa biến động của lợi suất ngày hợp đồng tương lai chỉ số VN30 kỳ hạn gần nhất (VN30F1M). Mã nguồn thực hiện ba bước:

1. **Bước 1 — so sánh mô hình trên dữ liệu thật.** Sáu mô hình (Bảng 4) được ước lượng trên tập huấn luyện bằng ba phương pháp: ước lượng hợp lý cực đại (MLE), Bayes biến phân mean-field (VB-MF) và Bayes biến phân full-rank (VB-FR). Chất lượng trong mẫu được đo bằng log-hợp lý, AIC, BIC; chất lượng ngoài mẫu đo bằng QLIKE, điểm log âm (NLS) và kiểm định VaR.
2. **Bước 2 — mô phỏng khôi phục tham số.** Dữ liệu được sinh từ EGARCH-X Student-t với tham số đã biết, rồi ước lượng lại $B = 100$ lần ở mỗi trường hợp E1–E3. Bước này kiểm tra bộ ước lượng có khôi phục đúng tham số, nhất là hệ số đòn bẩy θ, hay không.
3. **Bước 3 — MCMC.** Thuật toán NUTS lấy mẫu phân phối hậu nghiệm của mô hình thắng cuộc ở Bước 1. Mẫu MCMC là chuẩn tham chiếu để đánh giá độ rộng hậu nghiệm mà VB báo cáo.

1.2 Nguyên tắc cài đặt

- **Một hàm log-hợp lý cho ba phương pháp.** Các hàm `garch_loglik` và `egarch_loglik` được MLE tối ưu trực tiếp và được đưa vào mô hình Bayes qua `pyro.factor`.
- **Một nguồn sự thật cho danh mục mô hình.** Tệp `mo_hinh/danh_muc.py` định nghĩa sáu mô hình; script, báo cáo và kiểm thử đều đọc từ đây.
- **Mọi con số trong báo cáo đọc từ tệp kết quả.** Bảng và hình được sinh bằng script từ thư mục `results/`; script `kiem_toan_ket_qua.py` tính lại toàn bộ để đối chiếu.
- **Chạy lại được và ngắt được.** Mỗi cặp (mô hình, phương pháp) và mỗi lần lặp mô phỏng được lưu thành một tệp riêng; chạy lại sẽ bỏ qua phần đã có. Số ngẫu nhiên được cố định bằng hạt giống (seed); mọi phép tính dùng số thực 64 bit.

2 Cài đặt và cách chạy

2.1 Yêu cầu phần mềm và thư viện

Mã nguồn chạy trên Python 3.11 trở lên. Bảng 1 liệt kê các thư viện (tệp `requirements.txt`) và vai trò của chúng.

Bảng 1: Thư viện sử dụng

Thư viện	Phiên bản	Vai trò trong dự án
PyTorch (`torch`)	≥ 2.0	Tính toán tensor số thực 64 bit, vi phân tự động (autograd) cho gradient và Hessian của log-hợp lý, bộ tối ưu Adam và L-BFGS.
Pyro (`pyro-ppl`)	≥ 1.9	Ngôn ngữ lập trình xác suất trên PyTorch [24]: khai báo mô hình Bayes, suy diễn biến phân ngẫu nhiên (SVI) với `AutoNormal`/`AutoMultivariateNormal`, lấy mẫu NUTS.
NumPy, pandas	$\geq$ 1.26, 2.0	Đọc và gộp dữ liệu phút thành dữ liệu ngày, tính biến ngoại sinh, thống kê trên mảng.
SciPy	≥ 1.11	Phân vị và hàm phân phối t, chuẩn, χ^2 cho VaR và các kiểm định.
statsmodels, arch	$\geq$ 0.14, 6.0	Kiểm định ban đầu: ADF, KPSS, Ljung-Box, ARCH-LM, Jarque-Bera (statsmodels), Phillips-Perron (arch).
ArviZ	≥ 0.16	Chẩn đoán hội tụ MCMC: $\hat{R}$ và cỡ mẫu hiệu dụng (ESS).
Matplotlib	≥ 3.7	Vẽ hình cho báo cáo.
pytest	≥ 7.0	Kiểm thử tự động.

2.2 Cài đặt

```
conda create -n garch python=3.11
conda activate garch
cd "code thuật toán"
pip install -r requirements.txt
python -m pytest tests -q               # 43 kiểm thử, khoảng 1 phút
python scripts/kiem_toan_ket_qua.py     # đối chiếu 380 con số, vài phút
```

Trên Windows có thể mở tệp `chay.bat`: tệp này tự tìm trình thông dịch Python có `torch` và `pyro`, rồi hiện một bảng chọn cho các thao tác ở Mục 2.3. Thư viện tự chuyển đầu ra cửa sổ lệnh sang UTF-8 khi được nạp để in được tiếng Việt.

2.3 Chạy từng bước

Mọi lệnh chạy từ thư mục `code thuật toán`. Mỗi script có cờ `--nhanh` dùng cấu hình rất nhỏ để thử đường chạy trong vài phút; kết quả của cờ này ghi vào thư mục riêng (`results/pipeline_nhanh/`, `results/mo_phong_egarch_nhanh/`) nên không lẫn với kết quả thật.

Bảng 2: Các lệnh chạy chính; mỗi tệp chạy bằng `python scripts/<tệp>`

Tệp (và tùy chọn)	Tác dụng và đầu ra
`chay_buoc1_so_sanh_mo_hinh.py`	Bước 1. Tạo `results/00_data.pt`, `01_preliminary_tests.pt`; với mỗi mô hình và phương pháp tạo `results/pipeline/<mô hình>_<phương pháp>.pt`; in bảng so sánh và lưu `bang_so_sanh.pt`. Tùy chọn: `--kiem-ke` (chỉ liệt kê việc sẽ chạy), `--chi-bang` (chỉ in bảng).
`chay_buoc2_mo_phong.py --jobs 8`	Bước 2. Chạy song song 900 lần ước lượng (3 trường hợp × 3 phương pháp × 100 lần lặp), mỗi lần lặp một tệp trong `results/mo_phong_egarch/parts/`; gộp thành `E1.pt`, `E2.pt`, `E3.pt`. Tùy chọn: `--truong-hop E2`, `--B`, `--gop`.
`chay_buoc3_mcmc.py`	Bước 3. Chọn mô hình có QLIKE nhỏ nhất trong `bang_so_sanh.pt` (hoặc chỉ định bằng `--mo-hinh egarch_x_t`), chạy 4 chuỗi NUTS, lưu `results/pipeline/mcmc_<mô hình>.pt`.
`kiem_toan_ket_qua.py`	Tính lại mọi con số từ nguồn thô và đối chiếu với tệp kết quả.
`bao_cao/ve_vector_tham_so.py`	Vẽ hình trung bình tích lũy của ước lượng qua các lần lặp mô phỏng.
`bao_cao/sinh_hinh_tong_hop.py`	Vẽ các hình minh họa (giá, lợi suất, AIC, θ, R_{sd}).
`bao_cao/sinh_tong_hop_ket_qua.py`	Sinh `Bài báo/tong_hop_ket_qua.tex`; biên dịch bằng `xelatex` hai lần.

Thời gian chạy ở cấu hình đầy đủ, ghi trong trường `giay` của các tệp kết quả, được liệt kê ở Bảng 3. Phần lớn thời gian nằm ở vòng lặp phương sai: mỗi lần đánh giá log-hợp lý là một vòng lặp tuần tự qua 1.735 quan sát, và mỗi bước tối ưu hoặc mỗi bước leapfrog của NUTS cần một lần tính xuôi và một lần lan truyền ngược qua vòng lặp đó.

Bảng 3: Thời gian chạy ghi trong các tệp kết quả

Mô hình	MLE (phút)	VB-MF (phút)	VB-FR (phút)
GARCH chuẩn	13,6	8,2	11,1
GARCH Student-t	13,7	8,3	11,0
GARCH-X Student-t	14,6	10,1	12,7
EGARCH chuẩn	33,2	18,8	23,5
EGARCH Student-t	39,7	24,0	27,5
EGARCH-X Student-t	34,0	20,0	25,0

Bước 2: tổng 357 giờ tính toán (một nhân) cho 900 lần ước lượng, khoảng 30 giờ trên 12 tiến trình.
Bước 3: 8,1 giờ cho 4 chuỗi NUTS chạy tuần tự trên EGARCH-X Student-t.

3 Cấu trúc thư mục và luồng dữ liệu

```
code thuật toán/
  ohlc_export.csv            dữ liệu phút VN30F1M (đầu vào duy nhất)
  requirements.txt           danh sách thư viện
  chay.bat                   bảng chọn thao tác trên Windows
  garch_vn30/                THƯ VIỆN
    du_lieu/                 doc_du_lieu.py, chuan_bi.py
    kiem_dinh/               kiem_dinh_so_bo.py
    mo_hinh/                 phan_phoi.py, garch.py, egarch.py, danh_muc.py
    suy_dien/                mle.py, bayes.py, vb.py, mcmc.py
    danh_gia/                ham_mat_mat.py, kiem_dinh_var.py,
                             kiem_dinh_dm.py, ngoai_mau.py
    mo_phong/                sinh_du_lieu.py, truong_hop.py, khoi_phuc.py
  scripts/                   ĐIỀU PHỐI
    chung.py                 đường dẫn, lưu checkpoint, chạy song song
    chay_buoc1_so_sanh_mo_hinh.py, chay_buoc2_mo_phong.py,
    chay_buoc3_mcmc.py, kiem_toan_ket_qua.py
    bao_cao/                 sinh_tong_hop_ket_qua.py, sinh_hinh_tong_hop.py,
                             ve_vector_tham_so.py
  tests/                     kiểm thử (pytest)
  results/                   kết quả (xem Mục 6)
  tai_lieu/                  tài liệu này
```

Luồng dữ liệu đi theo một chiều: `ohlc_export.csv` → `du_lieu` tạo chuỗi r_t, RV_t, X_{t-1} → `suy_dien` ước lượng tham số trên tập huấn luyện → `danh_gia` tính chỉ tiêu ngoài mẫu → tệp `.pt` trong `results/` → `scripts/bao_cao` sinh bảng và hình. Nhánh mô phỏng thay bước đọc dữ liệu thật bằng `mo_phong/sinh_du_lieu.py`. Thư viện không đọc hay ghi tệp kết quả nào; mọi thao tác với tệp nằm trong `scripts/`, nhờ vậy các hàm của thư viện được kiểm thử độc lập.

4 Cơ sở lý thuyết

Mục này trình bày các công thức được cài đặt theo đúng thứ tự xử lý. Tên hàm cài đặt được ghi trong ngoặc.

4.1 Dữ liệu và biến số

Tệp nguồn chứa nến 1 phút của chuỗi hợp đồng liên tục VN30F1M do nhà cung cấp ghép sẵn. Gọi $P_{t,j}$ là giá đóng cửa của nến phút thứ j trong ngày t và P_t là giá đóng cửa của nến cuối ngày. Lợi suất log ngày tính theo phần trăm và biến động thực tế (realized volatility) là

$$r_t = 100\,(\log P_t - \log P_{t-1}), \qquad RV_t = 100^2 \sum_{j \geq 2} \big(\log P_{t,j} - \log P_{t,j-1}\big)^2. \tag{1}$$

RV_t chỉ cộng lợi suất giữa các nến trong cùng một ngày, không gồm khoảng qua đêm; nó là đại diện quan sát được cho phương sai ngày, cùng đơn vị $\%^2$ với phương sai có điều kiện h_t (`load_vn30f1m_daily`).

Vector biến ngoại sinh gồm hai biến trễ một phiên, tức chỉ dùng thông tin đã biết trước ngày t:

$$X_{t-1} = \big(\log(1 + VOL_{t-1}),\ \log(RV_{t-1} + 10^{-6})\big)^{\top}, \tag{2}$$

với VOL là tổng khối lượng giao dịch trong ngày. Mỗi biến được chuẩn hóa $\tilde{x} = (x - \bar{x}_{\text{train}})/s_{\text{train}}$, trong đó trung bình và độ lệch chuẩn chỉ tính trên tập huấn luyện để thông tin của tập kiểm định không rò rỉ vào mô hình (`standardize_with_train_stats`). Dữ liệu được chia theo thời gian, không xáo trộn: 80% phiên đầu làm tập huấn luyện (1.735 quan sát), phần còn lại làm tập kiểm định (435 quan sát) (`chuan_bi_du_lieu`).

4.2 Phân phối sai số

Mọi mô hình có dạng $r_t = \mu + \varepsilon_t$, $\varepsilon_t = \sqrt{h_t}\, z_t$, với z_t độc lập cùng phân phối, $\mathbb{E}z_t = 0$ và $\operatorname{Var} z_t = 1$. Hai lựa chọn cho z_t là phân phối chuẩn $N(0,1)$ và phân phối Student-t chuẩn hóa t_ν^* với mật độ

$$f(z \mid \nu) = \frac{\Gamma\!\left(\frac{\nu+1}{2}\right)}{\Gamma\!\left(\frac{\nu}{2}\right)\sqrt{\pi(\nu-2)}} \left(1 + \frac{z^2}{\nu - 2}\right)^{-\frac{\nu+1}{2}}, \qquad \nu > 2. \tag{3}$$

So với Student-t thông thường, t_ν^* được co tỷ lệ bởi $\sqrt{(\nu-2)/\nu}$ để có phương sai bằng 1; nhờ đó h_t luôn là phương sai có điều kiện của r_t với cả hai phân phối. Bậc tự do ν càng nhỏ thì đuôi càng dày; khi $\nu \to \infty$, t_ν^* hội tụ về $N(0,1)$. Hằng số chuẩn hóa được tính qua hàm $\log\Gamma$ để không tràn số (`std_t_logpdf`, `std_t_log_const`).

EGARCH cần kỳ vọng của $|z|$:

$$\mathbb{E}|z| = \sqrt{2/\pi}\ \text{(chuẩn)}, \qquad \mathbb{E}|z| = \frac{2\sqrt{\nu-2}\ \Gamma\!\left(\frac{\nu+1}{2}\right)}{(\nu-1)\sqrt{\pi}\ \Gamma\!\left(\frac{\nu}{2}\right)}\ \text{(Student-}t\text{ chuẩn hóa)}. \tag{4}$$

Công thức thứ hai suy từ $\mathbb{E}|T| = 2\sqrt{\nu}\,\Gamma(\frac{\nu+1}{2})/[(\nu-1)\sqrt{\pi}\,\Gamma(\frac{\nu}{2})]$ của biến $T \sim t_\nu$ và $z = T\sqrt{(\nu-2)/\nu}$ (`expected_abs_std_t`). Một kiểm thử riêng xác nhận giá trị này hội tụ về $\sqrt{2/\pi} \approx 0{,}7979$ khi ν lớn.

Để mô phỏng, $z \sim t_\nu^*$ được sinh bằng $z = \dfrac{Z}{\sqrt{V/\nu}}\sqrt{\dfrac{\nu-2}{\nu}}$ với $Z \sim N(0,1)$ và $V \sim \chi^2_\nu$ độc lập (`std_t_sample`).

4.3 Mô hình GARCH(1,1) và GARCH-X

Phương sai có điều kiện theo Bollerslev [1], bổ sung biến ngoại sinh:

$$h_t = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,h_{t-1} + \gamma^{\top} X_{t-1}, \qquad \omega > 0,\ \alpha, \beta \geq 0. \tag{5}$$

α đo phản ứng của phương sai với cú sốc mới; β đo độ dai dẳng. Khi $\gamma = 0$ và $\alpha + \beta < 1$, phương sai dài hạn là $\omega/(1-\alpha-\beta)$. Do $\gamma^{\top} X_{t-1}$ có thể âm, vế phải có thể âm, nên mã nguồn cắt dưới $h_t \geq 10^{-8}$ (hằng số `H_MIN`). Điều kiện đầu h_0 là phương sai mẫu của chuỗi dùng để ước lượng. Các số hạng ε_{t-1}^2 và $\gamma^{\top} X_{t-1}$ không phụ thuộc h nên được tính một lần ngoài vòng lặp (`garch_forward`).

4.4 Mô hình EGARCH(1,1) và EGARCH-X

EGARCH của Nelson [2] viết phương trình cho logarit phương sai:

$$\log h_t = \omega + \alpha\big(|z_{t-1}| - \mathbb{E}|z|\big) + \theta\, z_{t-1} + \beta \log h_{t-1} + \gamma^{\top} X_{t-1}, \qquad z_{t-1} = \frac{\varepsilon_{t-1}}{\sqrt{h_{t-1}}}. \tag{6}$$

- α là hiệu ứng độ lớn: cú sốc có trị tuyệt đối lớn hơn kỳ vọng làm tăng biến động.
- θ là hiệu ứng dấu, hay hiệu ứng đòn bẩy: với $\theta < 0$, cú sốc âm $z_{t-1} = -c$ làm $\log h_t$ tăng thêm $(\alpha - \theta)c$, lớn hơn mức $(\alpha + \theta)c$ của cú sốc dương cùng độ lớn.
- β là độ dai dẳng; quá trình $\log h_t$ dừng khi $|\beta| < 1$, và khi đó $\mathbb{E} \log h_t = \omega/(1-\beta)$ nếu $\mathbb{E}X = 0$.

Vì $h_t = \exp(\cdot)$ nên $h_t > 0$ theo cấu trúc, không cần ràng buộc dấu cho $\omega, \alpha, \theta, \gamma$. Mã nguồn chặn $\log h_t \in [-30, 30]$ chỉ để hàm mũ không tràn số; đây là lưới an toàn số học, không phải ràng buộc mô hình. Khác với GARCH, z_{t-1} phụ thuộc h_{t-1} vừa tính nên mọi số hạng phải tính trong vòng lặp (`egarch_forward`). Đề tài quy ước $\beta \in (0, 1)$ cho cả ba phương pháp vì tiên nghiệm Bayes của β là phân phối Beta trên $(0, 1)$; như vậy ba phương pháp làm việc trên cùng một không gian tham số.

4.5 Hàm log-hợp lý có điều kiện

Với vector tham số ϑ và f_z là mật độ chuẩn hoặc (3), đổi biến $z_t = (r_t - \mu)/\sqrt{h_t}$ cho log-hợp lý có điều kiện

$$\ell(\vartheta) = \sum_{t=1}^{T-1} \Big[\log f_z\Big(\frac{r_t - \mu}{\sqrt{h_t(\vartheta)}} \Big) - \tfrac{1}{2} \log h_t(\vartheta) \Big], \tag{7}$$

trong đó $-\frac{1}{2}\log h_t$ là Jacobian của phép đổi biến và quan sát $t = 0$ chỉ dùng làm điều kiện đầu (`log_mat_do`, `garch_loglik`, `egarch_loglik`). Mọi phép tính viết bằng tensor PyTorch nên gradient $\nabla \ell$ và Hessian $\nabla^2 \ell$ được tính chính xác bằng vi phân tự động, không dùng sai phân hữu hạn.

4.6 Danh mục mô hình

Bảng 4: Sáu mô hình so sánh; d là số tham số tự do với $k = 2$ biến ngoại sinh

Mã (code)	Tên	Họ	Phân phối	Có X	θ	d
`garch_n`	GARCH chuẩn	GARCH	chuẩn	không	không	4
`garch_t`	GARCH Student-t	GARCH	Student-t	không	không	5
`garch_x_t`	GARCH-X Student-t	GARCH	Student-t	có	không	7
`egarch_n`	EGARCH chuẩn	EGARCH	chuẩn	không	có	5
`egarch_t`	EGARCH Student-t	EGARCH	Student-t	không	có	6
`egarch_x_t`	EGARCH-X Student-t	EGARCH	Student-t	có	có	8

Hai mô hình kề nhau trong danh mục chỉ khác đúng một yếu tố (phân phối, biến ngoại sinh, hoặc họ phương trình), nên chênh lệch chỉ tiêu giữa chúng cho biết đóng góp của yếu tố đó. Công thức $d = 4 + \mathbf{1}_{\text{EGARCH}} + k\,\mathbf{1}_X + \mathbf{1}_t$ nằm trong `ModelSpec.n_params`.

4.7 Ước lượng hợp lý cực đại

Tham số hóa không ràng buộc. MLE tìm $\hat{\vartheta} = \arg\max \ell(\vartheta)$ trên miền hợp lệ. Để dùng thuật toán tối ưu không ràng buộc, mỗi tham số được viết qua một biến $\eta \in \mathbb{R}$ (`unpack`, `pack`):

GARCH	EGARCH
$\omega = e^{\eta_\omega}$, $\alpha = \text{softplus}(\eta_\alpha)$, $\beta = \text{softplus}(\eta_\beta)$	ω, α, θ tự do, $\beta = \sigma(\eta_\beta) = 1/(1+e^{-\eta_\beta})$
Cả hai họ: μ, γ tự do; $\nu = 2 + \text{softplus}(\eta_\nu)$, $\text{softplus}(x) = \log(1+e^x)$.	

Nhiều điểm khởi tạo. Log-hợp lý của mô hình GARCH có thể có nhiều cực đại địa phương. Hàm `fit_mle` chạy từ `n_starts` điểm khởi tạo ngẫu nhiên (seed $0, 1, 2, \ldots$) quanh một cấu hình hợp lý: với EGARCH, $\beta \approx 0{,}9$ và $\omega \approx (1-\beta)\log h_0$ để mức nền của $\log h$ khớp dữ liệu; θ khởi tạo quanh 0 để không giả định trước dấu đòn bẩy. Nghiệm được chọn là nghiệm có log-hợp lý hữu hạn lớn nhất. Cấu hình mặc định của Bước 1 là 10 điểm khởi tạo; kết quả đang lưu của ba mô hình GARCH được tính ở một lần chạy trước với 6 điểm khởi tạo (số điểm khởi tạo thực tế lưu trong trường `runs`).

Giai đoạn 1: Adam. Từ mỗi điểm khởi tạo, thuật toán Adam [4] giảm $L(\eta) = -\ell$ với cập nhật

$$m_k = \beta_1 m_{k-1} + (1-\beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2, \quad \eta_{k+1} = \eta_k - \lambda \frac{m_k/(1-\beta_1^k)}{\sqrt{v_k/(1-\beta_2^k)} + \epsilon}, \tag{8}$$

với $g_k = \nabla L(\eta_k)$, tốc độ học $\lambda = 0{,}02$ và các hằng số mặc định của PyTorch ($\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$). Chuẩn gradient được cắt ở 50 để một bước lớn không đẩy tham số ra vùng số học bất ổn. Adam ổn định khi điểm hiện tại còn xa nghiệm.

Giai đoạn 2: L-BFGS. Gần nghiệm, thuật toán tựa Newton L-BFGS [5, 6] dùng hướng $p_k = -H_k \nabla L(\eta_k)$, trong đó H_k xấp xỉ nghịch đảo Hessian từ các cặp $(s_i, y_i) = (\eta_{i+1} - \eta_i, \nabla L_{i+1} - \nabla L_i)$ gần nhất, và độ dài bước a thỏa điều kiện Wolfe mạnh

$$L(\eta_k + a p_k) \le L(\eta_k) + c_1 a \nabla L_k^\top p_k, \qquad |\nabla L(\eta_k + a p_k)^\top p_k| \le c_2 |\nabla L_k^\top p_k|. \tag{9}$$

Nếu L-BFGS gặp giá trị không hữu hạn trong lúc tìm kiếm đường, mã nguồn quay về nghiệm của giai đoạn Adam. Một điểm khởi tạo được đánh dấu hội tụ khi log-hợp lý hữu hạn và $\|\nabla L\| < 1$ (lớp `LanChay`).

Tiêu chí thông tin. Với n quan sát huấn luyện và d tham số:

$$\text{AIC} = 2d - 2\ell(\hat{\vartheta}), \qquad \text{BIC} = d \log n - 2\ell(\hat{\vartheta}); \tag{10}$$

giá trị nhỏ hơn là tốt hơn. BIC phạt số tham số nặng hơn AIC khi $\log n > 2$.

Sai số chuẩn và khoảng Wald. Theo lý thuyết tiệm cận của MLE, $\hat{\eta}$ xấp xỉ phân phối chuẩn với hiệp phương sai bằng nghịch đảo Hessian quan sát $\mathcal{H} = \nabla^2 L(\hat{\eta})$. Phương pháp delta chuyển sang thang tự nhiên $\vartheta = g(\eta)$ (`standard_errors`):

$$\widehat{\text{Cov}}(\hat{\vartheta}) = J\, \mathcal{H}^{-1} J^\top, \qquad J = \left.\frac{\partial g}{\partial \eta}\right|_{\hat{\eta}}, \qquad \text{SE}_j = \sqrt{\left[J \mathcal{H}^{-1} J^\top\right]_{jj}}. \tag{11}$$

Cả $\mathcal{H}$ và J đều tính bằng vi phân tự động. Nếu $\mathcal{H}$ không khả nghịch hoặc cho phương sai âm, nghiệm không phải cực đại trong (thường do nằm sát biên), tiệm cận chuẩn không áp dụng và hàm trả về `None`. Khoảng tin cậy Wald 95% là $\hat{\theta}_j \pm 1{,}96\,\text{SE}_j$.

4.8 Mô hình Bayes và tiên nghiệm

Suy diễn Bayes dựa trên định lý Bayes, $p(\vartheta \mid r) \propto p(r \mid \vartheta)\, p(\vartheta)$, với $p(r \mid \vartheta) = \exp \ell(\vartheta)$ là cùng hàm hợp lý (7). Trong Pyro, mỗi tham số là một *site* lấy mẫu từ tiên nghiệm và log-hợp lý được cộng vào mật độ bằng `pyro.factor` (`bayes.model`). Tiên nghiệm (lớp `PriorConfig`) là tiên nghiệm yếu:

Tham số	Tiên nghiệm	Ghi chú
μ	$N(0,1)$	
ω (GARCH)	LogNormal$(0,1)$	bảo đảm $\omega > 0$
ω (EGARCH)	$N(0,1)$	không cần dương
$\alpha,\ \beta$	Beta$(2,2)$ trên $(0,1)$	
θ (EGARCH)	$N(0,1)$	đối xứng quanh 0, không giả định dấu đòn bẩy
γ_j	$N(0,1)$	
$\nu - 2$	Gamma$(2;\ 0{,}2)$	kỳ vọng của ν bằng 12

Pyro tự biến đổi mỗi site có miền giới hạn về $\mathbb{R}$ (log, logit, …) và cộng log-Jacobian của phép biến đổi vào mật độ; VB và NUTS làm việc trên không gian đã biến đổi này.

4.9 Bayes biến phân

VB xấp xỉ hậu nghiệm bằng một phân phối q_ϕ trong một họ cho trước, chọn tham số ϕ để cực đại cận dưới bằng chứng (ELBO) [7]:

$$\mathrm{ELBO}(\phi) = \mathbb{E}_{q_\phi}\big[\log p(r,\vartheta) - \log q_\phi(\vartheta)\big] = \log p(r) - \mathrm{KL}\big(q_\phi \,\|\, p(\cdot \mid r)\big) \le \log p(r). \tag{12}$$

Cực đại ELBO tương đương cực tiểu độ phân kỳ Kullback–Leibler từ q_ϕ tới hậu nghiệm. Gọi $\tilde{\vartheta} \in \mathbb{R}^d$ là tham số sau biến đổi. Hai họ xấp xỉ được cài đặt (`fit_vb`):

- **Mean-field** (`AutoNormal`): $q(\tilde{\vartheta}) = \prod_j N(m_j, s_j^2)$, bỏ qua mọi tương quan hậu nghiệm giữa các tham số.
- **Full-rank** (`AutoMultivariateNormal`): $q(\tilde{\vartheta}) = N(m, LL^\top)$ với L tam giác dưới, nắm được tương quan tuyến tính giữa các tham số.

Gradient của ELBO được ước lượng bằng mẹo tham số hóa lại [8]: $\tilde{\vartheta} = m + L\epsilon$, $\epsilon \sim N(0,I)$, nên $\nabla_\phi \mathrm{ELBO} = \mathbb{E}_\epsilon \nabla_\phi [\log p(r, \tilde{\vartheta}(\epsilon)) - \log q_\phi(\tilde{\vartheta}(\epsilon))]$ được xấp xỉ bằng một mẫu ϵ mỗi bước (`Trace_ELBO`). Bộ tối ưu Adam với tốc độ học $0{,}01$ chạy 3.000 bước ở Bước 1 và 1.500 bước ở Bước 2; quy trình này gọi là suy diễn biến phân ngẫu nhiên (SVI). ELBO báo cáo là trung bình 50 bước cuối. Sau khi khớp, mã nguồn rút 1.000 mẫu từ q (`Predictive`) để tính trung bình, độ lệch chuẩn và khoảng hậu nghiệm 2,5%–97,5% (`posterior_summary`). ELBO chỉ so sánh được giữa các lần khớp của cùng một mô hình; nó không thay AIC/BIC khi so sánh các mô hình khác nhau.

4.10 MCMC bằng NUTS

Hamiltonian Monte Carlo. HMC [9] coi $U(\tilde{\vartheta}) = -\log p(\tilde{\vartheta} \mid r)$ là thế năng, thêm biến động lượng $p \sim N(0, M)$ và mô phỏng hệ Hamilton $H = U(\tilde{\vartheta}) + \frac{1}{2} p^\top M^{-1} p$ bằng tích phân leapfrog với bước ε:

$$p \leftarrow p - \tfrac{\varepsilon}{2}\nabla U(\tilde{\vartheta}), \qquad \tilde{\vartheta} \leftarrow \tilde{\vartheta} + \varepsilon M^{-1} p, \qquad p \leftarrow p - \tfrac{\varepsilon}{2}\nabla U(\tilde{\vartheta}). \tag{13}$$

Điểm cuối quỹ đạo được chấp nhận với xác suất Metropolis $\min\{1, e^{-\Delta H}\}$. Nhờ dùng gradient, HMC di chuyển xa trong không gian tham số mà vẫn giữ xác suất chấp nhận cao.

NUTS. Thuật toán No-U-Turn Sampler [10] tự chọn độ dài quỹ đạo: quỹ đạo được nhân đôi dần cho tới khi bắt đầu quay đầu, tức $(\tilde{\vartheta}^+ - \tilde{\vartheta}^-)^\top p^\pm < 0$, hoặc đạt độ sâu cây tối đa (tối đa $2^{\text{độ sâu}}$ bước leapfrog mỗi mẫu). Trong giai đoạn khởi động (warmup), ε được thích nghi bằng trung bình kép (dual averaging) để xác suất chấp nhận trung bình đạt 0,85, và ma trận khối lượng M được ước lượng từ mẫu khởi động. Bước 3 dùng M đặc (`full_mass`) để nắm tương quan giữa các tham số, 4 chuỗi, mỗi chuỗi 1.000 vòng khởi động và 400 mẫu, độ sâu cây tối đa 7 (`fit_mcmc`). Số bước leapfrog phân kỳ (năng lượng tăng vọt) được đếm để báo cáo.

Khởi tạo chuỗi. Chuỗi 0 bắt đầu đúng tại nghiệm MLE; ba chuỗi còn lại được nhiễu loạn hai sai số chuẩn (11) quanh nghiệm này (`khoi_tao_tu_mle`). Tham số dương được nhiễu loạn trên thang log với độ lớn SE/x, tham số trong $(0,1)$ trên thang logit với độ lớn $\mathrm{SE}/[x(1-x)]$ (phương pháp delta), có trần 1,0 trên thang đã biến đổi. Các điểm xuất phát như vậy đủ phân tán để chẩn đoán $\hat{R}$ có ý nghĩa nhưng không đẩy chuỗi sang một vùng xa của không gian tham số.

Chẩn đoán hội tụ. Với m chuỗi, mỗi chuỗi tách đôi thành hai nửa độ dài n, gọi W là trung bình phương sai trong chuỗi và B là n lần phương sai giữa các trung bình chuỗi. Thống kê Gelman–Rubin [11]

$$\hat{R} = \sqrt{\frac{\frac{n-1}{n}W + \frac{1}{n}B}{W}} \tag{14}$$

gần 1 khi các chuỗi trộn tốt. ArviZ tính phiên bản chuẩn hóa xếp hạng của $\hat{R}$ và cỡ mẫu hiệu dụng khối $\mathrm{ESS} = mn/(1 + 2\sum_k \hat{\rho}_k)$ theo Vehtari và cộng sự [12]; ngưỡng thường dùng là $\hat{R} < 1{,}01$. Để đánh giá VB, tỷ số độ lệch chuẩn hậu nghiệm

$$R_{sd,j} = \mathrm{sd}_{\mathrm{VB}}(\theta_j)/\mathrm{sd}_{\mathrm{MCMC}}(\theta_j) \tag{15}$$

nhỏ hơn 1 cho thấy VB báo bất định thấp hơn MCMC (`sd_ratio`).

4.11 Đánh giá ngoài mẫu

Thiết kế. Tham số được ước lượng một lần trên tập huấn luyện. Vòng lặp phương sai sau đó chạy tiếp qua tập kiểm định với tham số cố định, điều kiện đầu là phương sai mẫu của tập huấn luyện. Tại mỗi t của tập kiểm định, $\hat{h}_t$ chỉ dùng thông tin đến $t-1$, nên là dự báo một bước ngoài mẫu (`duong_h_va_logf`).

Hàm mất mát. Với n quan sát kiểm định (`ham_mat_mat.py`):

$$\mathrm{MAE} = \frac{1}{n}\sum_t |RV_t - \hat{h}_t|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_t (RV_t - \hat{h}_t)^2}, \tag{16}$$

$$\mathrm{QLIKE} = \frac{1}{n}\sum_t \left(\log \hat{h}_t + \frac{RV_t}{\hat{h}_t}\right), \qquad \mathrm{NLS} = -\frac{1}{n}\sum_t \log \hat{f}_t(r_t). \tag{17}$$

QLIKE giữ nguyên thứ tự xếp hạng các mô hình khi RV_t là đại diện không chệch nhưng có nhiễu của phương sai thật [13]. NLS đánh giá toàn bộ mật độ dự báo, gồm cả đuôi.

Dự báo từ hậu nghiệm. Với $S = 200$ bộ tham số $\vartheta^{(s)}$ rút cách đều từ mẫu hậu nghiệm VB (`rut_bo_tham_so`), dự báo phương sai là $\bar{h}_t = S^{-1}\sum_s h_t^{(s)}$, và mật độ dự báo là hỗn hợp

$$\hat{f}_t(r) = \frac{1}{S}\sum_{s=1}^{S} f\left(r \mid \vartheta^{(s)}\right), \qquad \log \hat{f}_t(r) = \mathrm{logsumexp}_s \log f\left(r \mid \vartheta^{(s)}\right) - \log S, \tag{18}$$

tính ổn định bằng logsumexp (`danh_gia_hau_nghiem`). Qua kênh này, bất định tham số đi vào dự báo mật độ.

Giá trị chịu rủi ro. VaR mức $a \in \{1\%, 5\%\}$ của vị thế mua và chỉ báo vi phạm là

$$\mathrm{VaR}_t(a) = -\left(\mu + \sqrt{\hat{h}_t}\, q_a\right), \qquad I_t = \mathbf{1}\{r_t < -\mathrm{VaR}_t(a)\}, \tag{19}$$

với q_a là phân vị mức a của z: $\Phi^{-1}(a)$ cho phân phối chuẩn, $t_\nu^{-1}(a)\sqrt{(\nu-2)/\nu}$ cho Student-t chuẩn hóa (`phan_vi_z`). Với dự báo hậu nghiệm, VaR là trung bình phân vị của các thành phần.

Kiểm định Kupiec. Với x vi phạm trong n phiên và $\hat{\pi} = x/n$, kiểm định bao phủ vô điều kiện [14] có giả thuyết $H_0 : \Pr(I_t = 1) = a$ và thống kê

$$\mathrm{LR}_{uc} = -2\big[(n-x)\log(1-a) + x\log a - (n-x)\log(1-\hat{\pi}) - x\log\hat{\pi}\big] \;\sim\; \chi^2_1. \tag{20}$$

Kiểm định Christoffersen. Gọi n_{ij} là số lần chuyển từ trạng thái i sang j của chuỗi I_t, $\hat{\pi}_{01} = n_{01}/(n_{00}+n_{01})$, $\hat{\pi}_{11} = n_{11}/(n_{10}+n_{11})$ và $\hat{\pi}$ là tỷ lệ vi phạm chung. Kiểm định độc lập [15] so sánh xích Markov bậc một với mô hình độc lập:

$$\mathrm{LR}_{ind} = -2\log\frac{(1-\hat{\pi})^{n_{00}+n_{10}}\hat{\pi}^{n_{01}+n_{11}}}{(1-\hat{\pi}_{01})^{n_{00}}\hat{\pi}_{01}^{n_{01}}(1-\hat{\pi}_{11})^{n_{10}}\hat{\pi}_{11}^{n_{11}}} \sim \chi^2_1, \qquad \mathrm{LR}_{cc} = \mathrm{LR}_{uc} + \mathrm{LR}_{ind} \sim \chi^2_2. \tag{21}$$

Bác bỏ H_0 của LR_{ind} nghĩa là các vi phạm có xu hướng xảy ra thành cụm.

Kiểm định Diebold–Mariano. Để so sánh hai mô hình theo một hàm mất mát, với $d_t = L_{1t} - L_{2t}$ [16]:

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{\hat{S}/n}}, \qquad \hat{S} = \hat{\gamma}_0 + 2\sum_{k=1}^{h-1}\Big(1-\frac{k}{h}\Big)\hat{\gamma}_k, \tag{22}$$

với $\hat{\gamma}_k$ là hiệp phương sai trễ k của d_t (trọng số Newey–West [18]); hiệu chỉnh mẫu nhỏ của Harvey, Leybourne và Newbold [17] nhân DM với $\sqrt{[n+1-2h+h(h-1)/n]/n}$ và so với phân phối t_{n-1} (`diebold_mariano_test`). $\mathrm{DM} > 0$ cho thấy mô hình 1 có mất mát trung bình lớn hơn. Hàm này được cài đặt và kiểm thử nhưng chưa dùng trong các bảng hiện có.

4.12 Kiểm định thống kê ban đầu

Trước khi mô hình hóa, chuỗi r_t được kiểm tra bằng các kiểm định sau (`kiem_dinh_so_bo.py`):

- **ADF** [19] và **Phillips–Perron** [20]: hồi quy $\Delta r_t = c + \rho r_{t-1} + \ldots$; H_0: $\rho = 0$ (có nghiệm đơn vị). PP hiệu chỉnh tự tương quan và phương sai thay đổi bằng phương pháp phi tham số.
- **KPSS** [21]: đảo chiều giả thuyết, H_0: chuỗi dừng quanh mức.
- **Ljung–Box** [22]: $Q(m) = n(n+2)\sum_{k=1}^{m}\hat{\rho}_k^2/(n-k) \sim \chi^2_m$ trên r_t (tự tương quan của lợi suất) và trên r_t^2 (tự tương quan của biến động).
- **ARCH-LM** [3]: hồi quy e_t^2 lên $e_{t-1}^2, \ldots, e_{t-q}^2$; $\mathrm{LM} = nR^2 \sim \chi^2_q$; bác bỏ H_0 nghĩa là có hiệu ứng ARCH, cơ sở để dùng mô hình GARCH.
- **Jarque–Bera** [23]: $\mathrm{JB} = \frac{n}{6}\big[S^2 + \frac{1}{4}(K-3)^2\big] \sim \chi^2_2$ với độ lệch S và độ nhọn K; bác bỏ H_0 phân phối chuẩn gợi ý dùng sai số đuôi dày.

4.13 Nghiên cứu mô phỏng khôi phục tham số

Cơ chế sinh dữ liệu. Với tham số thật ϑ_0, hàm `simulate_egarch` sinh $X_t \sim N(0, I_2)$ độc lập, $z_t \sim t^*_{\nu_0}$, đặt $\log h_0 = \omega/(1-\beta)$, rồi lặp (6) và $r_t = \mu + \sqrt{h_t}z_t$. Mô phỏng sinh $T + 200$ quan sát và bỏ 200 quan sát đầu (burn-in) để xóa ảnh hưởng của điều kiện đầu. Lần lặp b dùng hạt giống b cho cả sinh dữ liệu và ước lượng, nên từng lần lặp tái lập được, không phụ thuộc thứ tự chạy song song.

Ba trường hợp. Các tham số chung là $\mu = 0{,}05$, $\omega = 0{,}04$, $\alpha = 0{,}15$, $\beta = 0{,}95$, $\gamma = (0{,}05; 0{,}05)$; khi đó $\omega/(1-\beta) = 0{,}8$, tức $h \approx 2{,}2$, cùng bậc với phương sai mẫu của VN30F1M. Các trường hợp khác nhau ở θ_0 và ν_0 (`truong_hop_mac_dinh`):

Trường hợp	θ_0	ν_0	Câu hỏi kiểm tra
E1	$-0{,}10$	8	Bộ ước lượng có khôi phục được hiệu ứng đòn bẩy không?
E2	0	8	Khi dữ liệu không có đòn bẩy, $\hat{\theta}$ có lệch khỏi 0 một cách hệ thống không?
E3	$-0{,}10$	4	Khôi phục có còn đúng khi đuôi rất dày, gần vùng $\hat{\nu}$ của VN30F1M?

Mỗi trường hợp có $B = 100$ lần lặp với $T = 2000$. MLE dùng 3 điểm khởi tạo, VB dùng 1.500 bước (`CAU_HINH_MAC_DINH`).

Chỉ tiêu. Với tham số j và ước lượng $\hat{\theta}_{j,b}$ (`tong_hop`):

$$\text{Bias}_j = \frac{1}{B}\sum_b (\hat{\theta}_{j,b} - \theta_{0j}), \quad \text{RMSE}_j = \sqrt{\frac{1}{B}\sum_b (\hat{\theta}_{j,b} - \theta_{0j})^2}, \quad \text{SE}_{MC,j} = \sqrt{\text{RMSE}_j^2 - \text{Bias}_j^2}, \tag{23}$$

$$\text{Coverage}_j = \frac{1}{B'}\sum_b \mathbf{1}\{L_{j,b} \le \theta_{0j} \le U_{j,b}\}. \tag{24}$$

$[L, U]$ là khoảng hậu nghiệm 2,5%–97,5% với VB và khoảng Wald với MLE; với MLE chỉ tính trên $B' \le B$ lần lặp có Hessian xác định dương. Coverage gần 0,95 cho thấy khoảng 95% được hiệu chỉnh tốt. Sai số Monte Carlo của coverage ở $B = 100$ vào khoảng $\sqrt{0{,}95 \cdot 0{,}05/100} \approx 0{,}022$.

5 Mô tả chi tiết từng tệp và hàm

Mỗi bảng dưới đây liệt kê các hàm công khai của một tệp; hàm có tên bắt đầu bằng dấu gạch dưới là hàm phụ trợ nội bộ.

5.1 Gói `garch_vn30` (`__init__.py`)

Đặt kiểu số thực mặc định của PyTorch là 64 bit (vòng lặp qua 1.700 quan sát và phép nghịch đảo Hessian cần độ chính xác cao) và chuyển đầu ra sang UTF-8. Mỗi gói con có `__init__.py` xuất lại các hàm chính.

5.2 `du_lieu/doc_du_lieu.py` — đọc dữ liệu phút

Hàm / lớp	Tác dụng và kỹ thuật
`load_intraday_csv(path)`	Đọc CSV, ghép cột thời gian từ ngày và giờ, đổi tên cột giá và khối lượng, sắp xếp theo thời gian.
`data_quality_report(df)`	Báo cáo chất lượng: số dòng trùng thời điểm, giá trị thiếu, giá hoặc khối lượng không dương, nến vi phạm $Low \le \min(O,C) \le \max(O,C) \le High$, số nến mỗi ngày.
`build_daily_bars(df)`	Gộp nến phút thành nến ngày: mở cửa đầu ngày, cao nhất, thấp nhất, đóng cửa cuối ngày, tổng khối lượng, số nến.
`compute_daily_log_returns`	$r_t = 100 \log(P_t/P_{t-1})$; ngày đầu là NaN.
`compute_intraday_log_returns`	Lợi suất log giữa hai nến phút liên tiếp trong cùng ngày; nến đầu ngày là NaN để không nối qua đêm.
`load_vn30f1m_daily(path)`	Hàm tổng hợp: trả về bảng ngày có cột `log_return` và `RV` theo (1), kèm báo cáo chất lượng.

5.3 `du_lieu/chuan_bi.py` — chuẩn bị dữ liệu mô hình

Hàm / lớp	Tác dụng và kỹ thuật
`build_exogenous_matrix(daily)`	Hai biến ngoại sinh trễ một phiên: $\log(1 + VOL_{t-1})$, $\log(RV_{t-1} + 10^{-6})$ (phép `shift(1)` của pandas).
`standardize_with_train_stats`	Chuẩn hóa $(x - \bar{x})/s$ với $\bar{x}, s$ chỉ tính trên các dòng của tập huấn luyện.
`chuan_bi_du_lieu(csv, train_frac)`	Toàn bộ khâu chuẩn bị: đọc, tạo biến, đặt mặt nạ huấn luyện 80% trên các phiên gốc rồi mới lọc dòng thiếu; trả về dict `r_all`, `X_all`, `rv_all`, `dates_all`, `n_train` (nội dung `00_data.pt`).

5.4 `kiem_dinh/kiem_dinh_so_bo.py` — kiểm định ban đầu

Hàm / lớp	Tác dụng và kỹ thuật
`adf_test`, `pp_test`, `kpss_test`	Kiểm định nghiệm đơn vị và tính dừng (gọi statsmodels và arch); trả về thống kê, p-value và kết luận ở mức 5%.
`ljung_box_test(x, lags)`	Thống kê $Q(m)$ của Ljung–Box đến độ trễ m.
`arch_lm_test(resid, nlags)`	Kiểm định ARCH-LM của Engle.
`jarque_bera_test(x)`	Kiểm định phân phối chuẩn, kèm độ lệch và độ nhọn.
`run_all_preliminary_tests(r)`	Chạy toàn bộ: ADF, PP, KPSS, Ljung-Box trên r và r^2, ARCH-LM, JB (nội dung `01_preliminary_tests.pt`).

5.5 `mo_hinh/phan_phoi.py` — phân phối sai số

Hàm / lớp	Tác dụng và kỹ thuật
`std_t_log_const(nu)`, `std_t_logpdf(z, nu)`	Hằng số chuẩn hóa và log-mật độ của t_ν^* theo (3), qua `lgamma` và `log1p` để ổn định số học; hỗ trợ broadcasting.
`normal_logpdf(z)`	Log-mật độ $N(0,1)$.
`expected_abs_normal()`, `expected_abs_std_t(nu)`	$\mathbb{E}\lvert z\rvert$ theo (4), dùng trong EGARCH.
`std_t_sample(nu)`	Sinh mẫu t_ν^* từ thương giữa biến chuẩn và căn biến khi bình phương.

5.6 `mo_hinh/garch.py` và `mo_hinh/egarch.py` — phương trình mô hình

Hàm / lớp	Tác dụng và kỹ thuật
`GARCHParams`, `EGARCHParams`	Bộ tham số dạng `NamedTuple` bất biến: `mu, omega, alpha, (theta,) beta, gamma, nu`; `nu=None` nghĩa là phân phối chuẩn.
`garch_forward(params, r, X, h0)`	Đường phương sai (5) với cắt dưới 10^{-8}; các số hạng không phụ thuộc h tính trước vòng lặp.
`egarch_forward(params, r, X, h0)`	Đường phương sai (6), chặn $\log h \in [-30, 30]$; trả về h (không phải $\log h$) để dùng chung giao diện với GARCH.
`log_mat_do(r, mu, h, nu)`	Log-mật độ có điều kiện từng quan sát, gồm số hạng Jacobian $-\frac{1}{2}\log h_t$; dùng chung cho hai họ và cho đánh giá ngoài mẫu.
`garch_loglik`, `egarch_loglik`	Log-hợp lý (7), khả vi theo mọi tham số.

Hàm / lớp	Tác dụng và kỹ thuật
`expected_abs_z(nu)`	Chọn công thức $\mathbb{E}\lvert z\rvert$ theo phân phối.

5.7 `mo_hinh/danh_muc.py` — danh mục mô hình

Hàm / lớp	Tác dụng và kỹ thuật
`ModelSpec`	Đặc tả một mô hình: tên, mã, họ, phân phối, có dùng X hay không; phương thức `n_params(k)` trả về d.
`CATALOGUE`, `get(code)`	Bộ sáu mô hình (Bảng 4) và hàm tra theo mã, báo lỗi rõ khi sai tên.
`lay_X(spec, X_all)`	Ma trận ngoại sinh cho mô hình; mô hình không dùng X nhận ma trận 0 cột (khi đó γ là vector rỗng).
`forward_cua`, `loglik_cua`	Chọn hàm tính theo họ mô hình.
`tham_so_sang_dict`, `tham_so_tu_dict`	Chuyển bộ tham số sang dict tensor để lưu tệp và ngược lại; nhờ đó tệp kết quả không phụ thuộc tên lớp Python.

5.8 `suy_dien/mle.py` — ước lượng hợp lý cực đại

Hàm / lớp	Tác dụng và kỹ thuật
`unpack(raw, dist, family)`, `pack`	Biến đổi giữa không gian không ràng buộc và miền hợp lệ (Mục 4.7).
`fit_mle(r, X, dist, family, n_starts, ...)`	MLE nhiều điểm khởi tạo: Adam (cắt chuẩn gradient) rồi L-BFGS với tìm kiếm đường Wolfe mạnh; chọn nghiệm có log-hợp lý lớn nhất; trả về `MLEResult` gồm tham số, log-hợp lý, AIC, BIC, d và chẩn đoán từng điểm khởi tạo (`LanChay`).
`standard_errors(params, r, X, ...)`	Sai số chuẩn theo (11): Hessian quan sát và Jacobian bằng `torch.autograd.functional`; trả về `None` nếu Hessian không hợp lệ.

5.9 `suy_dien/bayes.py` — mô hình Bayes

Hàm / lớp	Tác dụng và kỹ thuật
`PriorConfig`	Siêu tham số tiên nghiệm (bảng ở Mục 4.8).
`model(r, X, spec, prior)`	Mô hình sinh Pyro: lấy mẫu các site theo tiên nghiệm, cộng log-hợp lý bằng `pyro.factor`. VB và NUTS đều gọi hàm này.
`params_tu_mau`, `rut_bo_tham_so`	Chuyển mẫu hậu nghiệm thành danh sách bộ tham số (rút cách đều S bộ) cho đánh giá ngoài mẫu.
`posterior_summary(samples)`	Trung bình, độ lệch chuẩn, phân vị 2,5% và 97,5% của từng site.
`sd_ratio`	Tỷ số R_{sd} giữa VB và MCMC.

5.10 `suy_dien/vb.py` và `suy_dien/mcmc.py`

Hàm / lớp	Tác dụng và kỹ thuật
`fit_vb(r, X, spec, guide_type, ...)`	SVI với guide mean-field hoặc full-rank (Mục 4.9); trả về `VBResult`: ELBO theo bước, ELBO cuối, mẫu và tóm tắt hậu nghiệm.

Hàm / lớp	Tác dụng và kỹ thuật
`fit_mcmc(r, X, spec, n_chains, ...)`	NUTS (Mục 4.10); các chuỗi chạy tuần tự để tránh lỗi tạo tiến trình trên Windows; tính $\hat{R}$, ESS, số phân kỳ; có thể lưu tiến độ định kỳ vào tệp vì một chuỗi có thể chạy nhiều giờ.
`khoi_tao_tu_mle(params, spec, chain, se)`	Điểm xuất phát của chuỗi quanh nghiệm MLE theo sai số chuẩn.
`inference_data(post)`	Đóng gói mẫu cho ArviZ, chọn cú pháp theo phiên bản ArviZ.

5.11 `danh_gia/` — đánh giá ngoài mẫu

Hàm / lớp	Tác dụng và kỹ thuật
`mae_vol`, `rmse_vol`, `qlike`, `negative_log_score`	Các hàm mất mát ở Mục 4.11 (`ham_mat_mat.py`).
`phan_vi_z`, `kupiec_test`, `christoffersen_test`	Phân vị VaR và hai kiểm định hậu kiểm (`kiem_dinh_var.py`).
`diebold_mariano_test`	Kiểm định DM với phương sai Newey–West và hiệu chỉnh HLN (`kiem_dinh_dm.py`).
`duong_h_va_logf`	$\hat{h}_t$ và $\log \hat{f}_t(r_t)$ trên tập kiểm định cho một bộ tham số (`ngoai_mau.py`).
`danh_gia_mot_diem`	Đánh giá dự báo từ một bộ tham số (MLE): MAE, RMSE, QLIKE, NLS, số vi phạm và kiểm định VaR 1%, 5%.
`danh_gia_hau_nghiem`	Đánh giá từ S bộ tham số hậu nghiệm với mật độ hỗn hợp.

5.12 `mo_phong/` — nghiên cứu mô phỏng

Hàm / lớp	Tác dụng và kỹ thuật
`generate_exogenous`, `simulate_egarch`	Sinh biến ngoại sinh và dữ liệu EGARCH-X với burn-in (`sinh_du_lieu.py`).
`TruongHop`, `truong_hop_mac_dinh`, `tham_so_that`, `THU_TU`	Định nghĩa ba trường hợp E1–E3 và danh sách tên tham số (`truong_hop.py`).
`chay_mot_lan_lap(method, r, X, spec, seed)`	Ước lượng một bộ dữ liệu mô phỏng bằng một phương pháp; trả về ước lượng điểm và khoảng 95% (hậu nghiệm với VB, Wald với MLE) (`khoi_phuc.py`).
`tong_hop(lan_lap, that)`	Gộp các lần lặp thành Bias, RMSE, Coverage.

5.13 Các script trong `scripts/`

Hàm / lớp	Tác dụng và kỹ thuật
`chung.py`	Đường dẫn chung; `luu` ghi tệp nguyên tử (ghi tệp tạm rồi đổi tên, mất điện giữa chừng không để lại tệp hỏng); `checkpoint` bỏ qua phần đã tính; `chay_song_song` chạy mỗi tác vụ trong một tiến trình con, mỗi tiến trình một luồng BLAS, ghi nhật ký vào `logs/`.

Hàm / lớp	Tác dụng và kỹ thuật
`chay_buoc1_so_sanh_mo_hinh.py`	`nap_du_lieu`, `chay_mle`, `chay_vb`, `uoc_luong_tat_ca`, `bang_so_sanh`, `in_bang`; cấu hình trong lớp `CauHinh` (MLE 10 điểm khởi tạo, 500 bước Adam, 250 bước L-BFGS; VB 3.000 bước, 1.000 mẫu, 200 bộ tham số cho đánh giá ngoài mẫu).
`chay_buoc2_mo_phong.py`	`worker` chạy một lần lặp; `gop` gộp các lần lặp và in tóm tắt θ; `main` lập danh sách lần lặp còn thiếu và điều phối song song.
`chay_buoc3_mcmc.py`	`chon_mo_hinh` theo QLIKE, `chay_mcmc` với khởi tạo quanh MLE.
`kiem_toan_ket_qua.py`	Tính lại log-hợp lý, AIC, BIC, chỉ tiêu ngoài mẫu từ tham số và mẫu đã lưu; tính lại Bias, RMSE, Coverage từ từng lần lặp; tính lại tóm tắt và $\hat{R}$ của MCMC; báo `DAT`/`HONG`.
`bao_cao/*.py`	Sinh `tong_hop_ket_qua.tex` và các hình từ `results/`; hình vector được đối chiếu với cột Bias trước khi lưu.

6 Hướng dẫn đọc mã nguồn

6.1 Quy ước chú thích trong mã

Mỗi tệp của thư viện mở đầu bằng một chuỗi mô tả (docstring) gồm các phần:

- **LÝ THUYẾT / MỤC ĐÍCH**: tệp làm gì, công thức toán nào được cài đặt, ý nghĩa của từng tham số;
- **THƯ VIỆN**: các hàm của thư viện bên ngoài được dùng trong tệp và vai trò của chúng.

Mỗi hàm có docstring nêu tác dụng, công thức (nếu có), **Tham số** (ý nghĩa, kiểu, kích thước tensor) và **Trả về**. Các dòng quan trọng có chú thích ngắn ở cuối dòng, ví dụ

`loss.backward()  # vi phân tự động: tính dL/d(eta)`.

Công thức trong chú thích viết bằng ký tự thường (ví dụ `h_t = omega + ...`) và trùng với công thức ở Mục 4 của tài liệu này. Có thể xem docstring ngay trong Python bằng `help(ten_ham)`.

6.2 Thứ tự đọc đề nghị

1. `mo_hinh/phan_phoi.py` → `garch.py` → `egarch.py`: phân phối sai số, phương trình phương sai và hàm log-hợp lý — lõi toán học của đề tài.
2. `mo_hinh/danh_muc.py`: sáu mô hình và các hàm tiện ích dùng chung.
3. `suy_dien/mle.py`: tham số hóa không ràng buộc, tối ưu Adam + L-BFGS, sai số chuẩn.
4. `suy_dien/bayes.py` → `vb.py` → `mcmc.py`: mô hình Bayes và hai cách suy diễn Bayes.
5. `danh_gia/ngoai_mau.py` (cùng `ham_mat_mat.py`, `kiem_dinh_var.py`): đánh giá ngoài mẫu.
6. `mo_phong/`: nghiên cứu mô phỏng.
7. `scripts/chay_buoc1_so_sanh_mo_hinh.py`: xem các hàm trên được ghép thành quy trình thế nào.

Các tệp trong `du_lieu/` và `kiem_dinh/` là tiền xử lý và kiểm định chuẩn, có thể đọc sau.

7 Các hàm thư viện bên ngoài được sử dụng

Bảng dưới đây giải thích từng hàm của thư viện bên ngoài xuất hiện trong mã, để người đọc không cần tra tài liệu của thư viện.

Hàm (thư viện)	Tác dụng trong dự án
PyTorch (`torch`) — tính toán tensor và vi phân tự động	
`torch.tensor`, `torch.zeros`, `torch.stack`, `torch.cat`	Tạo tensor (mảng số), ghép danh sách tensor thành một tensor.
`torch.set_default_dtype(torch.float64)`	Dùng số thực 64 bit cho mọi phép tính (đặt trong `garch_vn30/__init__.py`).
`x.requires_grad_(True)`, `loss.backward()`, `x.grad`	Vi phân tự động: đánh dấu biến cần đạo hàm; `backward` tính gradient của hàm mất mát theo các biến đó bằng quy tắc dây chuyền qua mọi phép toán đã thực hiện (kể cả vòng lặp phương sai); gradient lưu ở `x.grad`.
`torch.no_grad()`, `x.detach()`	Tắt việc theo dõi đạo hàm khi chỉ cần giá trị (đánh giá ngoài mẫu, lưu kết quả).
`torch.optim.Adam`	Bộ tối ưu Adam (Mục 4.7); `opt.zero_grad()` xóa gradient cũ, `opt.step()` cập nhật tham số.
`torch.optim.LBFGS(..., line_search_fn="strong_wolfe")`	Bộ tối ưu tựa Newton L-BFGS với tìm kiếm đường Wolfe mạnh; cần hàm `closure` tính lại hàm mất mát và gradient.
`torch.nn.utils.clip_grad_norm_`	Cắt chuẩn gradient về tối đa một ngưỡng (50) để tránh bước nhảy quá lớn.
`torch.nn.functional.softplus`, `torch.sigmoid`, `torch.exp`	Các phép biến đổi đưa tham số về miền hợp lệ.
`torch.autograd.functional.hessian`	Ma trận đạo hàm bậc hai chính xác của $-\ell$ (dùng cho sai số chuẩn).
`torch.autograd.functional.jacobian`	Ma trận Jacobian của phép biến đổi tham số (phương pháp delta).
`torch.linalg.inv`	Nghịch đảo ma trận Hessian.
`torch.lgamma`, `torch.log1p`, `torch.expm1`	$\log\Gamma(x)$, $\log(1+x)$, $e^x - 1$ — các dạng ổn định số học.
`torch.clamp`	Chặn giá trị trong một khoảng ($h_t \geq 10^{-8}$; $\log h_t \in [-30, 30]$).
`torch.logsumexp`	$\log\sum_s e^{a_s}$ tính ổn định; dùng cho mật độ dự báo hỗn hợp.
`torch.manual_seed`, `torch.Generator`	Cố định hạt giống ngẫu nhiên để tái lập.
`torch.distributions.Chi2`	Sinh biến khi bình phương khi mô phỏng Student-t.
`torch.save`, `torch.load`	Ghi và đọc tệp kết quả `.pt`.
Pyro (`pyro-ppl`) — lập trình xác suất	
`pyro.sample("ten", phân phối)`	Khai báo một tham số ngẫu nhiên (site) với tiên nghiệm cho trước.
`pyro.factor("loglik", l)`	Cộng log-hợp lý ℓ vào log-mật độ của mô hình.
`pyro.distributions`: `Normal`, `LogNormal`, `Beta`, `Gamma`	Các phân phối tiên nghiệm; `.expand([k]).to_event(1)` tạo vector k chiều độc lập.
`AutoNormal`, `AutoMultivariateNormal`	Tự tạo phân phối xấp xỉ q mean-field và full-rank cho VB.
`SVI(model, guide, optim, loss)`, `svi.step`	Vòng tối ưu ELBO; mỗi lần gọi `step` thực hiện một bước gradient và trả về $-$ELBO.
`Trace_ELBO`	Ước lượng ELBO và gradient bằng mẫu Monte Carlo với mẹo tham số hóa lại.
`pyro.optim.Adam`	Adam dùng trong SVI.
`Predictive(model, guide, num_samples)`	Rút mẫu tham số từ q sau khi khớp.
`NUTS`, `MCMC`, `mcmc.run`, `mcmc.get_samples`	Bộ lấy mẫu No-U-Turn và vòng chạy chuỗi Markov; lấy mẫu hậu nghiệm.

Hàm (thư viện)	Tác dụng trong dự án
`pyro.set_rng_seed`, `pyro.clear_param_store`	Cố định hạt giống; xóa tham số của lần khớp trước.
ArviZ, SciPy, statsmodels, arch, pandas, NumPy	
`arviz.from_dict`, `arviz.rhat`, `arviz.ess`	Đóng gói mẫu MCMC; tính $\hat{R}$ và cỡ mẫu hiệu dụng.
`scipy.stats.norm.ppf`, `t.ppf`, `chi2.cdf`	Phân vị chuẩn và Student-t (VaR), hàm phân phối χ^2 (giá trị p của Kupiec, Christoffersen).
`adfuller`, `kpss`, `acorr_ljungbox`, `het_arch`, `jarque_bera` (statsmodels); `PhillipsPerron` (arch)	Các kiểm định ban đầu.
`pandas.read_csv`, `to_datetime`, `groupby().agg`, `shift`, `merge`	Đọc dữ liệu phút, gộp theo ngày, dịch trễ một phiên, ghép bảng.
`numpy.log`, `mean`, `sqrt`	Tính toán trên mảng trong phần đánh giá và mô phỏng.
`matplotlib.pyplot`	Vẽ hình cho báo cáo.

8 Ví dụ: một lần chạy từ dữ liệu đến kết quả

Đoạn mã sau (chạy từ thư mục `code thuật toán`) dùng trực tiếp thư viện để ước lượng EGARCH-X Student-t bằng MLE và VB rồi đánh giá ngoài mẫu. Các script trong `scripts/` thực hiện đúng các bước này cho cả sáu mô hình, kèm lưu kết quả.

```
import torch
from garch_vn30.du_lieu import chuan_bi_du_lieu
from garch_vn30.mo_hinh import danh_muc
from garch_vn30.suy_dien import fit_mle, standard_errors, fit_vb
from garch_vn30.danh_gia import danh_gia_mot_diem

d = chuan_bi_du_lieu("ohlc_export.csv")         # 1. dữ liệu: r, X, RV, chia 80/20
n = d["n_train"]
spec = danh_muc.get("egarch_x_t")                # 2. chọn mô hình trong danh mục
X = danh_muc.lay_X(spec, d["X_all"])

res = fit_mle(d["r_all"][:n], X[:n], dist=spec.dist,      # 3. MLE trên tập huấn luyện
              family=spec.family, n_starts=10)
print(res.params.theta, res.aic, res.bic)        #    hệ số đòn bẩy, AIC, BIC
se = standard_errors(res.params, d["r_all"][:n], X[:n],
                     dist=spec.dist, family=spec.family)  # 4. sai số chuẩn
oos = danh_gia_mot_diem(res.params, spec.family, spec.dist,
                        d["r_all"], X, d["rv_all"], n)    # 5. ngoài mẫu
print(oos["qlike"], oos["nls"], oos["n_violations_1pct"])

vb = fit_vb(d["r_all"][:n], X[:n], spec,          # 6. VB mean-field
            guide_type="mean_field", n_steps=3000)
print(vb.summary["theta"])                       #    trung bình, sd, khoảng 95%
```

Diễn giải từng bước: (1) `chuan_bi_du_lieu` đọc 41 MB dữ liệu phút, trả về 2.170 lợi suất ngày cùng RV_t và hai biến ngoại sinh đã chuẩn hóa; (2) `get` lấy đặc tả mô hình, `lay_X` trả về ma trận ngoại sinh phù hợp; (3) `fit_mle` chạy 10 điểm khởi tạo, mỗi điểm Adam rồi L-BFGS, giữ nghiệm có log-hợp lý lớn nhất; (4) `standard_errors` tính Hessian bằng vi phân tự động và áp dụng phương pháp delta; (5) `danh_gia_mot_diem` chạy tiếp vòng lặp phương sai qua 435 phiên kiểm định và tính QLIKE, NLS, kiểm định VaR; (6) `fit_`

vb khớp phân phối xấp xỉ mean-field và trả về mẫu hậu nghiệm. Với cấu hình như trên, bước (3) mất khoảng nửa giờ trên một nhân CPU.

9 Định dạng tệp kết quả

Mọi tệp là tệp `torch.save` chứa dict, tensor và số Python; đọc bằng `torch.load(path, weights_only=False)`. Tệp không chứa đối tượng của lớp tự định nghĩa nên đọc được mà không phụ thuộc cấu trúc mã nguồn.

Tệp	Nội dung (khóa)
`results/00_data.pt`	`r_all` (T), `X_all` ($T \times 2$), `rv_all`, `dates_all`, `n_train`, `quality_report`, `exog_mean`, `exog_std`.
`results/01_preliminary_tests.pt`	Kết quả `run_all_preliminary_tests`.
`pipeline/<mô hình>_mle.pt`	`params` (dict tensor), `loglik`, `aic`, `bic`, `n_params`, `runs` (chẩn đoán từng điểm khởi tạo), `oos`, `giay`.
`pipeline/<mô hình>_vb_<guide>.pt`	`final_elbo`, `summary`, `samples` (1.000 mẫu mỗi site), `oos`, `giay`.
`oos` (trong hai loại trên)	`mae`, `rmse`, `qlike`, `nls`, `h_hat`, `n_test`, `kupiec_1pct`, `christoffersen_1pct`, `n_violations_1pct` và tương tự cho 5%.
`pipeline/bang_so_sanh.pt`	Bảng so sánh Bước 1 (danh sách dòng).
`pipeline/mcmc_egarch_x_t.pt`	`samples` (1.600 mẫu: 4 chuỗi nối tiếp), `summary`, `max_rhat`, `min_ess`, `n_divergences`, `giay`.
`mo_phong_egarch/parts/<TH>_<pp>_<b>.pt`	Một lần lặp: `uoc_luong`, `can_duoi`, `can_tren` (`None` nếu MLE không có Hessian hợp lệ), `b`, `giay`.
`mo_phong_egarch/<TH>.pt`	Với mỗi phương pháp: `bias`, `rmse`, `coverage`, `n_replications`, `n_co_khoang`, `tham_so_that`.

10 Tính tái lập và kiểm chứng

- **Hạt giống cố định.** MLE dùng seed $0, 1, \ldots$ cho các điểm khởi tạo; VB và MCMC đặt `pyro.set_rng_seed`; mô phỏng dùng seed bằng chỉ số lần lặp. Chạy lại cùng lệnh cho cùng kết quả trên cùng phiên bản thư viện.
- **Kiểm thử tự động.** 43 kiểm thử trong `tests/` kiểm tra: mật độ t_ν^* khớp SciPy; $\mathbb{E}|z|$ hội tụ về $\sqrt{2/\pi}$ và khớp Monte Carlo; log-hợp lý GARCH khớp một cài đặt NumPy viết tay; hiệu ứng đòn bẩy đúng dấu; tính dương của h_t trong EGARCH; `pack`/`unpack` là nghịch đảo của nhau; AIC, BIC đúng công thức; VB tạo đúng các site cho cả sáu mô hình; MCMC chạy và cho $\hat{R}$ hữu hạn; đánh giá hậu nghiệm với một mẫu trùng đánh giá một điểm; Kupiec, Christoffersen, DM đúng hướng; biến ngoại sinh được dịch trễ và chuẩn hóa chỉ bằng tập huấn luyện; mô phỏng tái lập theo seed; Bias, RMSE, Coverage đúng công thức.
- **Kiểm toán kết quả.** `kiem_toan_ket_qua.py` tính lại 380 con số của cả ba bước từ nguồn thô; lần chạy gần nhất cho 380 đạt, 0 hỏng.
- **Đối chiếu với phiên bản trước.** Thư viện hiện tại là bản viết lại sau khi bỏ phần mô hình chuyển trạng thái MS-GARCH-X. Trước khi thay thế, bản mới được chạy song song với bản cũ trên cùng dữ liệu và cùng seed: dữ liệu chuẩn bị, log-hợp lý, chỉ tiêu ngoài mẫu, nghiệm MLE, sai số chuẩn, mẫu VB, mẫu MCMC, dữ liệu mô phỏng và kết quả từng lần lặp đều trùng khớp (chênh lệch lớn nhất $4{,}4 \times 10^{-16}$, mức làm tròn số thực). Bản cũ, gồm cả phần MS-GARCH-X và kết quả của nó, được lưu ở nhánh `luu-tru/ms-garch-x` và thẻ `luu-tru-ms-garch-x` trên kho GitHub của dự án.

11 Tóm tắt kết quả hiện có

Chi tiết nằm trong `Bài báo/tong_hop_ket_qua.pdf`. Trên tập huấn luyện VN30F1M, EGARCH-X Student-t ước lượng bằng MLE có AIC 5606,87 và BIC 5650,54, thấp nhất trong sáu mô hình; trên tập kiểm định, cặp này cũng có QLIKE (0,9234) và NLS (1,5203) thấp nhất. NUTS trên mô hình này có max $\hat{R} = 1{,}0125$, ESS nhỏ nhất 1.340 trên 1.600 mẫu và không có bước phân kỳ. Các kết luận này giới hạn ở chuỗi VN30F1M, sáu đặc tả và cách chia mẫu đã dùng.

Tài liệu tham khảo

[1] T. Bollerslev (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.

[2] D. B. Nelson (1991). Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2), 347–370.

[3] R. F. Engle (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.

[4] D. P. Kingma, J. Ba (2015). Adam: A method for stochastic optimization. *ICLR*.

[5] D. C. Liu, J. Nocedal (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45, 503–528.

[6] J. Nocedal, S. J. Wright (2006). *Numerical Optimization*, 2nd ed. Springer.

[7] D. M. Blei, A. Kucukelbir, J. D. McAuliffe (2017). Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518), 859–877.

[8] D. P. Kingma, M. Welling (2014). Auto-encoding variational Bayes. *ICLR*.

[9] R. M. Neal (2011). MCMC using Hamiltonian dynamics. In *Handbook of Markov Chain Monte Carlo*, Chapman & Hall/CRC.

[10] M. D. Hoffman, A. Gelman (2014). The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15, 1593–1623.

[11] A. Gelman, D. B. Rubin (1992). Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4), 457–472.

[12] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, P.-C. Bürkner (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2), 667–718.

[13] A. J. Patton (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1), 246–256.

[14] P. H. Kupiec (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives*, 3(2), 73–84.

[15] P. F. Christoffersen (1998). Evaluating interval forecasts. *International Economic Review*, 39(4), 841–862.

[16] F. X. Diebold, R. S. Mariano (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263.

[17] D. Harvey, S. Leybourne, P. Newbold (1997). Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2), 281–291.

[18] W. K. Newey, K. D. West (1987). A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3), 703–708.

[19] D. A. Dickey, W. A. Fuller (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366), 427–431.

[20] P. C. B. Phillips, P. Perron (1988). Testing for a unit root in time series regression. *Biometrika*, 75(2), 335–346.

[21] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, Y. Shin (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54, 159–178.

[22] G. M. Ljung, G. E. P. Box (1978). On a measure of lack of fit in time series models. *Biometrika*, 65(2), 297–303.

[23] C. M. Jarque, A. K. Bera (1980). Efficient tests for normality, homoscedasticity and serial independence of regression residuals. *Economics Letters*, 6(3), 255–259.

[24] E. Bingham và cộng sự (2019). Pyro: Deep universal probabilistic programming. *Journal of Machine Learning Research*, 20(28), 1–6.